I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 4: Aprendizaje No-Supervisado

Joaquin Szterensus

30 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

Este informe presenta el análisis de clustering y la reducción de dimensionalidad aplicados a dos conjuntos de datos. En la sección de clustering, se implementaron los algoritmos K-means, Gaussian Mixture Model (GMM) y DBSCAN sobre el archivo `clustering.csv`. Para K-means y GMM se empleó el método de ganancias decrecientes (elbow method) a fin de determinar el número óptimo de clusters, y en DBSCAN se exploraron diferentes valores de ε y K para identificar regiones densas y puntos de ruido. En la parte de reducción de dimensionalidad, se aplicó Análisis de Componentes Principales (PCA) al dataset `MNIST_dataset.csv`, evaluando la relación entre el número de componentes y el error cuadrático medio de reconstrucción, y justificando la elección de una dimensión reducida adecuada.

1. Clustering de datos

1.1. Introducción

El objetivo de este ejercicio es agrupar un conjunto de datos no etiquetados en grupos (clusters) de elementos similares, de modo que los objetos dentro de cada grupo compartan características relevantes y se diferencien claramente de los de otros grupos. Para ello, se trabajó sobre el archivo `clustering.csv`, que contiene n muestras descritas por d variables continuas. La entrada del algoritmo es,entonces, una matriz de tamaño $n \times d$, donde cada fila representa una observación y cada columna una característica.

Como salida, cada algoritmo de clustering produce un vector de etiquetas de longitud n, asignando a cada muestra un identificador de cluster (un entero entre 1 y k en métodos como K-means y GMM, o valores de *core*, *border* y *noise* en DBSCAN). Estas etiquetas permiten visualizar la estructura subyacente de los datos, detectar grupos homogéneos y descubrir patrones sin necesidad de supervisión. En particular:

- Con K-means y GMM se busca particionar el espacio en k regiones convexas optimizando criterios de varianza entre clusters y probabilidad conjunta, respectivamente.
- Con DBSCAN se identifican grupos basados en densidades locales, distinguiendo regiones de alta densidad de puntos aislados o ruido.

1.2. Métodos

Para resolver el problema de clustering se implementaron tres clases desde cero en Python,utilizando unicamente la libreria `numpy`.

K-means. Sea $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$ un conjunto de datos, y k el número de clusters deseados. El algoritmo k-means busca particionar los datos en k grupos disjuntos $\{C_1, \ldots, C_k\}$ minimizando la siguiente función objetivo:

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \left\| \mathbf{x}_i - \boldsymbol{\mu}_j \right\|^2,$$

donde $\boldsymbol{\mu}_j$ es el centroide del cluster C_j, calculado como el promedio de los puntos asignados al mismo. Equivalentemente, si denotamos $c(i)$ como el índice del cluster asignado a $\mathbf{x}_i$, se puede expresar como

$$J = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{c(i)} \right\|^2.$$

El algoritmo se basa en un procedimiento iterativo de optimización alternada que consta de los siguientes pasos:

1. **Inicialización.** Se seleccionan k centroides iniciales. En la implementación utilizamos el método *k-means++*, que mejora

la estabilidad y la calidad de las soluciones iniciales al favorecer puntos alejados entre sí como posibles centroides.

2. **Paso de asignación.** Cada punto $\mathbf{x}_i$ se asigna al cluster cuyo centroide esté más cercano, es decir,

$$c(i) = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|.$$

3. **Paso de actualización.** Para cada cluster, se recalcula el centroide $\boldsymbol{\mu}_j$ como el promedio de los puntos asignados:

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i.$$

4. **Criterio de parada.** El algoritmo itera los pasos anteriores hasta que los centroides se estabilizan (i.e., cambian menos que una tolerancia predefinida ε) o se alcanza un número máximo de iteraciones.

La calidad de la partición resultante puede medirse mediante la *inercia entre clusters*, definida como la suma de distancias cuadradas entre cada punto y su centroide asignado. Si bien k-means no garantiza un mínimo global, suele producir buenos resultados prácticos, especialmente con una buena inicialización como k-means++.

Gaussian Mixture Model (GMM). Un modelo de mezclas gaussianas representa la distribución de los datos como una combinación ponderada de k distribuciones normales multivariadas. Formalmente, se modela la densidad conjunta como:

$$p(\mathbf{x}) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \Sigma_j),$$

donde π_j representa el peso del componente j (con $\sum_j \pi_j = 1$), $\boldsymbol{\mu}_j \in \mathbb{R}^d$ es la media del componente j, y $\Sigma_j \in \mathbb{R}^{d \times d}$ su matriz de covarianza. El modelo se ajusta mediante el algoritmo de *Máxima Verosimilitud* usando el procedimiento iterativo de *Expextation-Maximization* (EM), el cual alterna entre dos pasos:

1. **E-step (esperanza).** Se computa la probabilidad posterior de que cada punto $\mathbf{x}_i$ pertenezca al componente j, conocida como la *responsabilidad* γ_{ij}:

$$\gamma_{ij} = \frac{\pi_j \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)}{\sum_{l=1}^{k} \pi_l \, \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_l, \Sigma_l)}.$$

2. **M-step (maximización).** Se actualizan los parámetros del modelo usando las responsabilidades calculadas:

$$\begin{aligned} N_j &= \sum_{i=1}^{n} \gamma_{ij}, \\ \pi_j &= \frac{N_j}{n}, \\ \boldsymbol{\mu}_j &= \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} \mathbf{x}_i, \\ \Sigma_j &= \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^\top. \end{aligned}$$

El proceso se repite hasta convergencia, típicamente cuando la log-verosimilitud varía menos que un umbral de tolerancia.

Inicialización. Dado que EM es sensible a la inicialización, se usa K-means++ para estimar los centroides iniciales, que se utilizan como medias $\boldsymbol{\mu}_j$. Las matrices de covarianza se estiman por componente a partir de los datos asignados, y los pesos iniciales π_j se calculan como proporciones relativas.

Regularización. Para evitar singularidades numéricas (por ejemplo, covarianzas degeneradas), se suma un término de regularización εI a cada Σ_j.

DBSCAN. El algoritmo de DBSCAN es un metodo de clustering que agrupa puntos densamente conectados, sin requerir especificar el número de clusters a priori. Para cada punto $\mathbf{x}_i$, se define su vecindario como:

$$\mathcal{N}_\varepsilon(\mathbf{x}_i) = \{\mathbf{x}_j : \|\mathbf{x}_j - \mathbf{x}_i\| \leq \varepsilon\}.$$

El algoritmo clasifica los puntos en tres categorías:

- **Core:** si $|\mathcal{N}_\varepsilon(\mathbf{x}_i)| \geq$ min_samples, es un punto denso.
- **Border:** no es core, pero está en el vecindario de un core.
- **Ruido:** no es core ni border.

El procedimiento se resume así:

1. Para cada punto no visitado, se calcula su vecindario $\mathcal{N}_\varepsilon(\mathbf{x}_i)$.
2. Si no tiene suficientes vecinos, se marca como ruido. Si es core, se crea un nuevo cluster y se expanden los vecinos recursivamente.
3. La expansión incorpora tanto a los puntos core como a los border conectados al núcleo.

Ventajas: DBSCAN puede encontrar clusters de forma arbitraria y es robusto al ruido. No requiere especificar k y puede detectar automáticamente puntos atípicos.

Desventajas: Su desempeño depende de la elección de ε y `min_samples`, y puede tener dificultades en regiones de densidad variable.

Conjunto de datos y preprocesamiento. Dado que los algoritmos son sensibles a la escala, antes de entrenar se aplicó normalización *min-max* en cada dimensión, es decir, $\tilde{x}_{ij} = \dfrac{x_{ij} - \text{mín}_j}{\text{máx}_j - \text{mín}_j}$, llevando cada atributo al rango $[0, 1]$. No se aplicó aumento de datos ni se realizó una partición en conjuntos de entrenamiento y prueba tradicionales.

Métricas de rendimiento. Para comparar las configuraciones de los distintos algoritmos se utilizaron métricas internas específicas según el método:

- *DBSCAN:* se utilizó el **Silhouette score** penalizado por la proporción de puntos clasificados como ruido. Para los puntos no-ruido $\mathbf{x}_i$, se calcula

  $$s(i) = \frac{b(i) - a(i)}{\text{máx}\{a(i), b(i)\}},$$

  donde $a(i)$ es la distancia media a puntos de su mismo cluster y $b(i)$ es la menor distancia media a puntos de otro cluster. Luego, se promedia $s(i)$ sobre los puntos válidos para obtener $\bar{s}$, y se penaliza según la proporción de puntos útiles:

  $$\text{score penalizado} = \bar{s} \cdot \left(\frac{N_{\text{útiles}}}{N_{\text{total}}}\right)$$

  donde $N_{\text{útiles}}$ es la cantidad de puntos no clasificados como ruido.

- *K-means:* se utilizó la **inercia total**, definida como la suma de distancias cuadradas entre cada punto y su centroide asignado:

  $$\sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|^2,$$

  donde z_i es el índice del cluster al que fue asignado el punto $\mathbf{x}_i$. Esta métrica se graficó en función de k para aplicar el método del codo.

Selección de hiperparámetros.

- **K-means:** se evaluaron valores de $k \in \{2, \ldots, 20\}$ y se seleccionó el óptimo utilizando el *método del codo*, analizando la inercia total L en función de k. Se eligió el valor a partir del cual la inercia deja de disminuir significativamente (ver Figura 1).

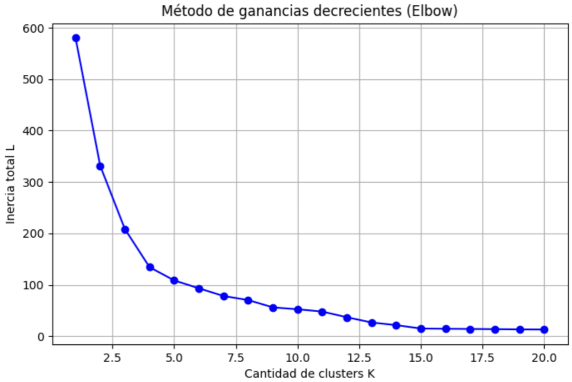


Figura 1: Método del codo para K-means: evolución de la inercia total en función de k.

- **GMM:** se aplicó un procedimiento similar, evaluando la log-verosimilitud total para distintos valores de k. Se utilizó también el método del codo para identificar el punto de saturación en la ganancia de log-verosimilitud (ver Figura 2).

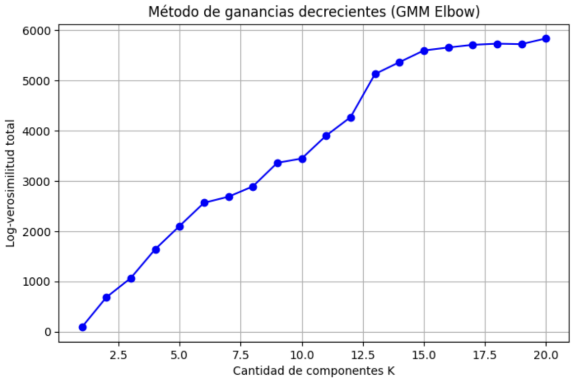


Figura 2: Método del codo para GMM: evolución de la log-verosimilitud total en función de k.

- **DBSCAN:** se realizó un *grid search* sobre $\varepsilon \in$ `np.linspace(0.01, 0.1, 20)` y min_samples $\in$ `np.linspace(5, 25, 10)`, seleccionando la configuración con mayor valor medio del coeficiente de Silhouette. Los parámetros óptimos fueron $\varepsilon \approx 0{,}024$ y min_samples $= 20$, con un Silhouette score de aproximadamente 0,59.

Para ambos modelos basados en k, el valor elegido fue $k = 15$.

1.3. Resultados

En esta sección se presentan y analizan los resultados obtenidos al aplicar diferentes métodos de clustering: DBSCAN, GMM y K-means, sobre el mismo conjunto de datos.

DBSCAN: En la Figura 3, se observa cómo la elección de ε y `min_samples` afecta notablemente el resultado del clustering. Valores muy pequeños de ε producen una gran cantidad de puntos clasificados como ruido, con muchos clusters pequeños. Al aumentar ε , el algoritmo logra capturar mejor las estructuras reales del conjunto de datos, reduciendo la cantidad de ruido.

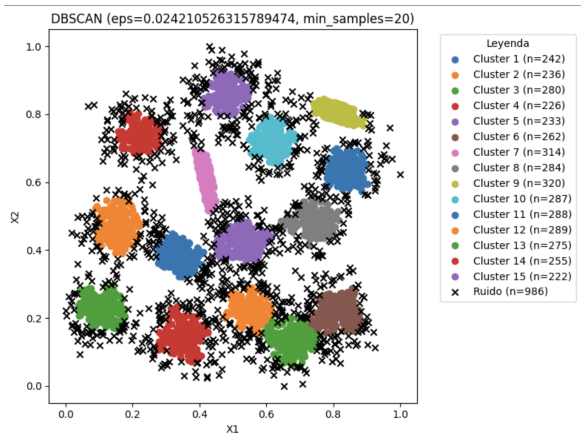


Figura 4: Mejor resultado de DBSCAN con $\varepsilon = 0{,}0242$ y `min_samples` $= 20$.

El mejor resultado de DBSCAN (Figura 4) se obtuvo con $\varepsilon = 0{,}0242$ y `min_samples` $= 20$, logrando identificar 15 clusters bien definidos, aunque con una cantidad considerable de puntos de ruido ($n = 986$). Esto indica que DBSCAN es eficaz para detectar la forma de los clusters, pero tiende a clasificar puntos periféricos como ruido. Estos hiperparametros pueden variar si se cambia la metrica utilizada, generando que el algoritmo genere asignaciones completamente nuevas.

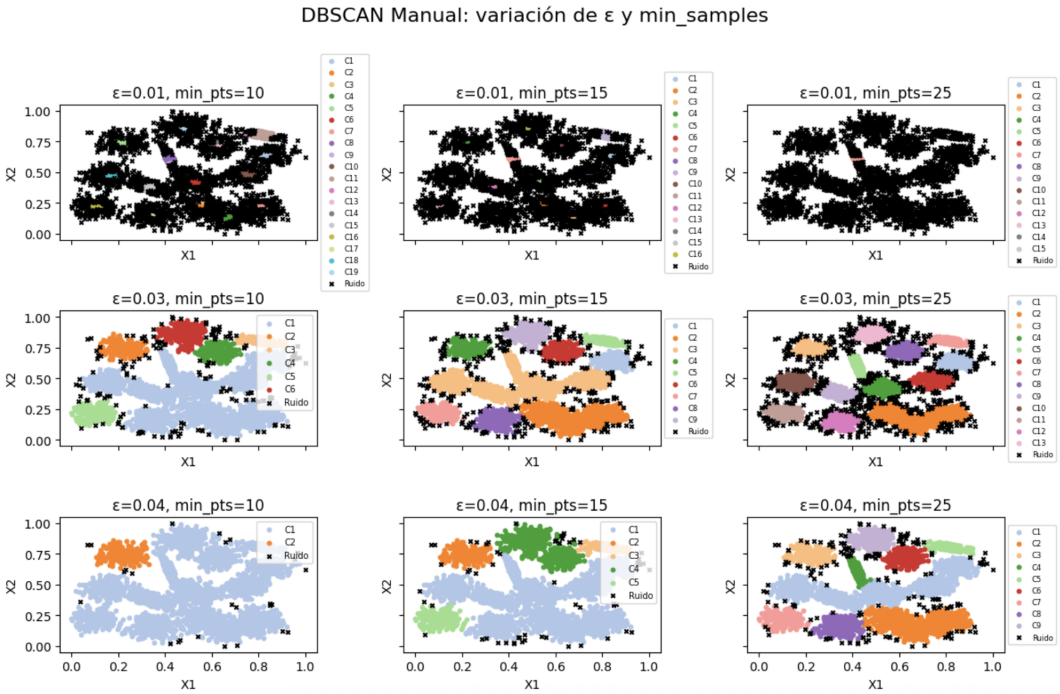
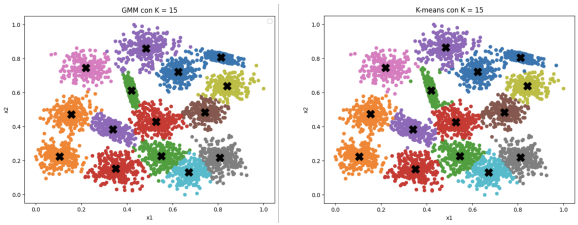


Figura 3: Variación del comportamiento de DBSCAN según los valores de ε y `min_samples`.



GMM con $K = 15$ K-means con $K = 15$

Figura 5: Resultados de GMM (izquierda) y K-means (derecha) con $K = 15$.

GMM y K-means: Ambos métodos funcionaron razonablemente bien cuando se fijó $K = 15$, tal como se observa en la Figura 5. GMM tiende a adaptarse mejor a formas no esféricas gracias al uso de distribuciones gaussianas, mientras que K-means, al asumir clusters esféricos, puede ser menos flexible. Sin embargo, ambos métodos lograron separar adecuadamente los grupos sin clasificar puntos como ruido.

Comparación general:

- **Mejor desempeño:** GMM y K-means presentaron resultados visualmente más claros y compactos, con todos los puntos asignados a algún cluster.

- **Peor desempeño:** DBSCAN con parámetros inadecuados, como $\varepsilon = 0{,}01$, generó demasiados clusters pequeños y una gran cantidad de ruido, lo cual no refleja la estructura real de los datos.

- **Ventaja de DBSCAN:** No requiere especificar el número de clusters y puede identificar estructuras más complejas, pero es muy sensible a sus parámetros.

En conclusión, si se conoce a priori el número de clusters, GMM o K-means son opciones sólidas. En cambio, DBSCAN puede ser más útil en situaciones donde se desconoce este valor o se espera detectar estructuras no lineales, siempre que se ajuste correctamente ε y `min_samples`.

2. Reducción de dimensionalidad

2.1. Introducción

El objetivo de esta sección es aplicar técnicas de reducción de dimensionalidad al conjunto de datos `MNIST_dataset.csv`, el cual contiene representaciones tabulares de imágenes de dígitos manuscritos del 0 al 9. Cada imagen es originalmente de 28×28 píxeles y se encuentra vectorizada en una fila con 784 características (una por cada píxel).

La entrada del algoritmo es entonces un conjunto de vectores de 784 dimensiones que representan imágenes de dígitos. Se utilizo como técnica **Principal Component Analysis (PCA)** para obtener representaciones de menor dimensión de los datos originales. La salida consiste en reconstrucciones de las imágenes a partir de estas representaciones comprimidas, con el fin de evaluar la calidad de la reducción.

Las técnicas de reducción de dimensionalidad y descomposición en valores singulares (SVD) son herramientas cruciales para el análisis y procesamiento de datos. Estas técnicas permiten simplificar conjuntos de datos de alta dimensionalidad, reduciendo el ruido y facilitando la visualización y análisis de la información. Su aplicación es particularmente relevante en campos como la inteligencia artificial, donde la eficiencia y precisión en el procesamiento de grandes volúmenes de datos son fundamentales, y en la compresión de imágenes, donde se busca reducir el tamaño de los archivos sin sacrificar la calidad visual.

2.2. Métodos

Para realizar la reducción de dimensionalidad del conjunto de datos, se implementó el algoritmo **Principal Component Analysis (PCA)** utilizando exclusivamente la biblioteca NumPy. El objetivo de PCA es encontrar una base ortogonal de menor dimensión que capture la mayor varianza posible del conjunto de datos original.

Dado un conjunto de datos $X \in \mathbb{R}^{n \times d}$, donde cada fila representa una imagen aplanada de 784 píxeles ($d = 784$), el primer paso consiste en centrar los datos restando la media de cada variable:

$$X_c = X - \mu, \quad \text{donde } \mu = \frac{1}{n} \sum_{i=1}^{n} X_i$$

Luego, se aplica la descomposición en valores singulares (SVD) a la matriz centrada X_c, obteniendo:

$$X_c = U \Sigma V^T$$

donde:

- $U \in \mathbb{R}^{n \times d}$ contiene los vectores ortonormales
- $\Sigma \in \mathbb{R}^{d \times d}$ es una matriz diagonal con los **valores singulares** $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_d \geq 0$,
- $V^T \in \mathbb{R}^{d \times d}$ contiene los **componentes principales** como vectores fila.

Los primeros k vectores de V se utilizan para proyectar los datos en un subespacio de menor dimensión:

$$Z = X_c \cdot W_k = X_c \cdot V_k^T$$

donde $W_k = V_k^T \in \mathbb{R}^{d \times k}$ representa los k primeros vectores principales. Esta matriz proyecta los datos originales al nuevo espacio de características reducido $Z \in \mathbb{R}^{n \times k}$.

La reconstrucción aproximada de los datos se realiza como:

$$\hat{X} = Z \cdot W_k^T + \mu$$

donde se vuelve a sumar la media original para deshacer el centrado previo.

Preprocesamiento: Antes de aplicar PCA, se normalizó cada pixel al rango $[0, 1]$ dividiendo los valores de intensidad por 255. Posteriormente, se centraron los datos restando la media de cada característica.

Selección de hiperparámetros: El parámetro principal a ajustar fue la cantidad de componentes principales k. Se calculó el error cuadrático medio de reconstrucción para valores de k entre 1 y 300, y se graficó esta relación para identificar el punto de saturación, donde agregar más componentes ya no disminuye significativamente el error. Este valor se tomó como referencia para elegir una representación comprimida eficiente.

Métricas de evaluación: La principal métrica utilizada fue el *Mean Squared Error* (MSE) entre las imágenes originales y las reconstruidas, definida como:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2$$

donde $x_i \in \mathbb{R}^d$ es la imagen original y $\hat{x}_i \in \mathbb{R}^d$ la reconstrucción obtenida a partir de su representación comprimida. Esta métrica permitió cuantificar cuánta información se pierde al reducir la dimensionalidad.

2.3. Resultados

2.3.1. Análisis de valores singulares

En la Figura 6 se muestran los valores singulares del conjunto de datos. Se observa una rápida caída inicial, lo que indica que gran parte de la varianza de los datos está contenida en las primeras componentes principales. A partir de cierto punto (aproximadamente $k > 250$), los valores singulares disminuyen lentamente, aportando poca varianza adicional, lo que sugiere que utilizar esa cantidad de dimensiones para la reconstrucción, debería resultar en un error extremadamente bajo.

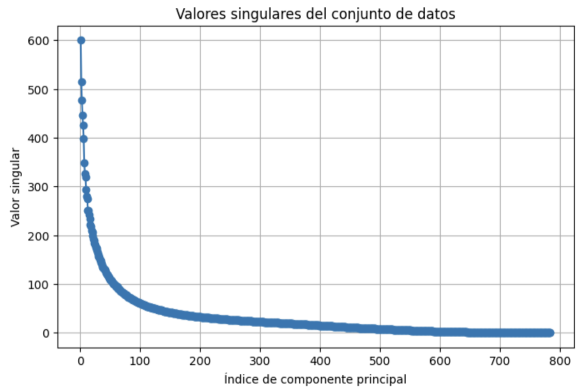


Figura 6: Valores singulares del conjunto de datos en función del índice de componente principal.

2.3.2. Error cuadrático medio

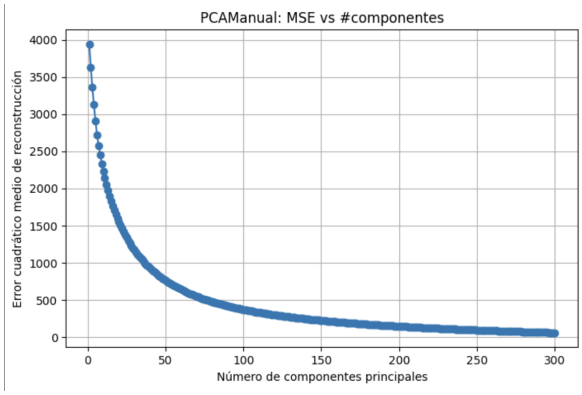


Figura 7: Error cuadrático medio de reconstrucción en función del número de componentes principales.

La Figura 7 muestra cómo varía el error cuadrático medio (MSE) de reconstrucción en función de la cantidad de componentes principales utilizadas. La curva decrece rápidamente al principio y luego se estabiliza, confirmando que las primeras decenas de componentes concentran la mayor parte de la información relevante. A partir de $k \approx 100$ se obtiene un compromiso adecuado entre compresión y calidad de reconstrucción.Como se menciono anteriormente, es posible ver como al llegar a las 250 dimensiones, el error casi que se reduce a

0, denotando en como las dimensiones restantes no aportan ningun tipo de informacion a la reconstruccion de la imagen

2.3.3. Reconstrucción de imágenes

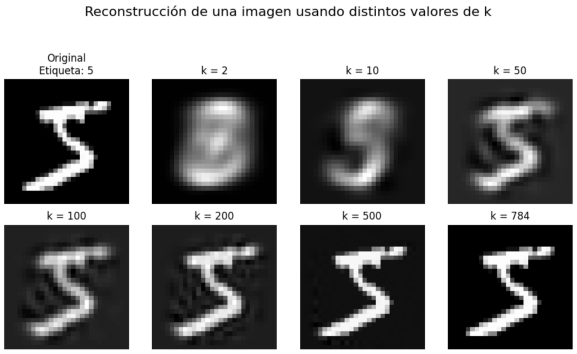


Figura 8: Reconstrucción de una imagen utilizando distintos valores de k.

En la Figura 8 se observa la reconstrucción de una imagen de dígito (5) utilizando distintas cantidades de dimensiones. Se aprecia que, con muy pocos componentes ($k = 2$ o $k = 10$), la imagen reconstruida es poco representativa, pero la calidad mejora notablemente al incrementar k. Con $k = 100$ ya se distingue claramente el dígito. Es evidente notar en las ultimas tres imágenes, las cuales son reconstrucciones con $k = 200$, $k = 500$ y $k = 784$ respectivamente, como la imagen modifica relativamente poco. Esto se debe que los nuevos valores singulares agregados al aumentar la dimensión de reconstrucción son muy bajos. Al utilizar la misma cantidad de dimensiones que la entrada, obtenemos que la imagen reconstruida es idéntica a la original.

2.3.4. Reconstrucción de múltiples imágenes

En la Figura 9 se muestran las reconstrucciones de una de cada una de las etiquetas del conjunto de datos, utilizando $k = 100$ componentes principales. En general, las imágenes

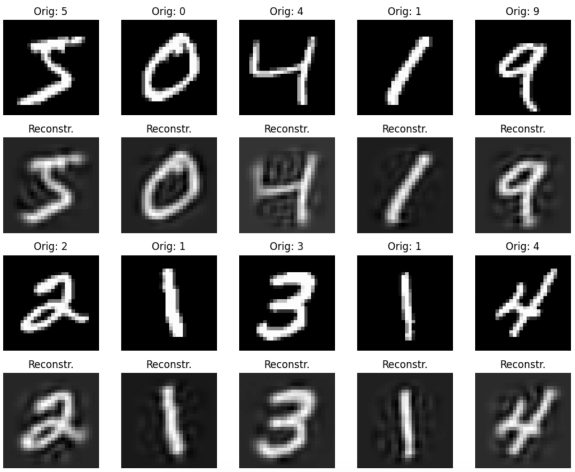


Figura 9: Reconstrucción de las primeras 10 imágenes del conjunto utilizando $k = 50$.

reconstruidas conservan la forma y trazos característicos de los dígitos originales, aunque se nota cierta pérdida de nitidez y definición.

2.4. Selección de k

En base al análisis visual y cuantitativo, se seleccionó $k = 100$ como valor adecuado para representar las imágenes comprimidas. Esta elección permite conservar la estructura general de los dígitos con un bajo costo computacional, y reduce significativamente la dimensionalidad original de 784 a 100 dimensiones.